

1. 拿不准的时候最好用括号来强制让表达式的组合关系符合程序逻辑的要求。
2. 如果改变了某个运算对象的值，在表达式的其他地方不要再使用这个运算对象。
- 第 2 条规则有一个重要例外，当改变运算对象的子表达式本身就是另外一个子表达式的运算对象时该规则无效。例如，在表达式*++iter 中，递增运算符改变 iter 的值，iter（已经改变）的值又是解引用运算符的运算对象。此时（或类似的情况下），求值的顺序不会成为问题，因为递增运算（即改变运算对象的子表达式）必须先求值，然后才轮到解引用运算。显然，这是一种很常见的用法，不会造成什么问题。

4.1.3 节练习

练习 4.3: C++语言没有明确规定大多数二元运算符的求值顺序，给编译器优化留下了余地。这种策略实际上是在代码生成效率和程序潜在缺陷之间进行了权衡，你认为这可以接受吗？请说出你的理由。

4.2 算术运算符

表 4.1: 算术运算符（左结合律）		
运算符	功能	用法
+	一元正号	+ expr
-	一元负号	- expr
*	乘法	expr * expr
/	除法	expr / expr
%	求余	expr % expr
+	加法	expr + expr
-	减法	expr - expr

表 4.1（以及后面章节的运算符表）按照运算符的优先级将其分组。一元运算符的优先级最高，接下来是乘法和除法，优先级最低的是加法和减法。优先级高的运算符比优先级低的运算符组合得更紧密。上面的所有运算符都满足左结合律，意味着当优先级相同时按照从左向右的顺序进行组合。

除非另做特殊说明，算术运算符都能作用于任意算术类型（参见 2.1.1 节，第 30 页）以及任意能转换为算术类型的类型。算术运算符的运算对象和求值结果都是右值。如 4.11 节（第 141 页）描述的那样，在表达式求值之前，小整数类型的运算对象被提升成较大的整数类型，所有运算对象最终会转换成同一类型。

140

一元正号运算符、加法运算符和减法运算符都能作用于指针。3.5.3 节（第 106 页）已经介绍过二元加法和减法运算符作用于指针的情况。当一元正号运算符作用于一个指针或者算术值时，返回运算对象值的一个（提升后的）副本。

一元负号运算符对运算对象值取负后，返回其（提升后的）副本：

```
int i = 1024;
int k = -i;      // k 是-1024
bool b = true;
bool b2 = -b;    // b2 是 true!
```

在 2.1.1 节（第 31 页），我们指出布尔值不应该参与运算，`-b` 就是一个很好的例子。

对大多数运算符来说，布尔类型的运算对象将被提升为 `int` 类型。如上所示，布尔变量 `b` 的值为真，参与运算时将被提升成整数值 1（参见 2.1.2 节，第 32 页），对它求负后的结果是 -1。将 -1 再转换回布尔值并将其作为 `b2` 的初始值，显然这个初始值不等于 0，转换成布尔值后应该为 1。所以，`b2` 的值是真！

提示：溢出和其他算术运算异常

算术表达式有可能产生未定义的结果。一部分原因是数学性质本身：例如除数是 0 的情况；另外一部分则源于计算机的特点：例如溢出，当计算的结果超出该类型所能表示的范围时就会产生溢出。

假设某个机器的 `short` 类型占 16 位，则最大的 `short` 数值是 32767。在这样一台机器上，下面的复合赋值语句将产生溢出：

```
short short_value = 32767; // 如果 short 类型占 16 位，则能表示的最大值是 32767
short_value += 1;          // 该计算导致溢出
cout << "short_value: " << short_value << endl;
```

给 `short_value` 赋值的语句是未定义的，这是因为表示一个带符号数 32768 需要 17 位，但是 `short` 类型只有 16 位。很多系统在编译和运行时都不报溢出错误，像其他未定义的行为一样，溢出的结果是不可预知的。在我们的系统中，程序的输出结果是：

```
short_value: -32768
```

该值发生了“环绕（wrapped around）”，符号位本来是 0，由于溢出被改成了 1，于是结果变成一个负值。在别的系统中也许会有其他结果，程序的行为可能不同甚至直接崩溃。

当作用于算术类型的对象时，算术运算符 `+`、`-`、`*`、`/` 的含义分别是加法、减法、乘法和除法。整数相除结果还是整数，也就是说，如果商含有小数部分，直接弃除：

```
int ival1 = 21/6;    // ival1 是 3，结果进行了删节，余数被抛弃掉了
int ival2 = 21/7;    // ival2 是 3，没有余数，结果是整数
```

运算符 `%` 俗称“取余”或“取模”运算符，负责计算两个整数相除所得的余数，参与取余运算的运算对象必须是整数类型： ◀ 141

```
int ival = 42;
double dval = 3.14;
ival % 12;          // 正确：结果是 6
ival % dval;        // 错误：运算对象是浮点类型
```

在除法运算中，如果两个运算对象的符号相同则商为正（如果不为 0 的话），否则商为负。C++ 语言的早期版本允许结果为负值的商向上或向下取整，C++11 新标准则规定商一律向 0 取整（即直接切除小数部分）。 C++
11

根据取余运算的定义，如果 `m` 和 `n` 是整数且 `n` 非 0，则表达式 `(m/n)*n+m%n` 的求值结果与 `m` 相等。隐含的意思是，如果 `m%n` 不等于 0，则它的符号和 `m` 相同。C++ 语言的早期版本允许 `m%n` 的符号匹配 `n` 的符号，而且商向负无穷一侧取整，这一方式在新标准中已经被禁止使用了。除了 -`m` 导致溢出的特殊情况，其他时候 `(-m)/n` 和 `m/(-n)` 都等于 `-(m/n)`，`m%(-n)` 等于 `m%n`，`(-m)%n` 等于 `-(m%n)`。具体示例如下：

```
21 % 6;      /* 结果是 3 */      21 / 6;      /* 结果是 3 */
21 % 7;      /* 结果是 0 */      21 / 7;      /* 结果是 3 */
-21 % -8;    /* 结果是 -5 */     -21 / -8;    /* 结果是 2 */
21 % -5;     /* 结果是 1 */      21 / -5;     /* 结果是 -4 */
```

142

4.2 节练习

练习 4.4： 在下面的表达式中添加括号，说明其求值的过程及最终结果。编写程序编译该（不加括号的）表达式并输出其结果验证之前的推断。

```
12 / 3 * 4 + 5 * 15 + 24 % 4 / 2
```

练习 4.5： 写出下列表达式的求值结果。

```
(a) -30 * 3 + 21 / 5      (b) -30 + 3 * 21 / 5
(c) 30 / 3 * 21 % 5      (d) -30 / 3 * 21 % 4
```

练习 4.6： 写出一条表达式用于确定一个整数是奇数还是偶数。

练习 4.7： 溢出是何含义？ 写出三条将导致溢出的表达式。

4.3 逻辑和关系运算符

关系运算符作用于算术类型或指针类型，逻辑运算符作用于任意能转换成布尔值的类型。逻辑运算符和关系运算符的返回值都是布尔类型。值为 0 的运算对象（算术类型或指针类型）表示假，否则表示真。对于这两类运算符来说，运算对象和求值结果都是右值。

表 4.2： 逻辑运算符和关系运算符			
结合律	运算符	功能	用法
右	!	逻辑非	!expr
左	<	小于	expr < expr
左	<=	小于等于	expr <= expr
左	>	大于	expr > expr
左	>=	大于等于	expr >= expr
左	==	相等	expr == expr
左	!=	不相等	expr != expr
左	&&	逻辑与	expr && expr
左		逻辑或	expr expr

逻辑与和逻辑或运算符

对于逻辑与运算符（&&）来说，当且仅当两个运算对象都为真时结果为真；对于逻辑或运算符（||）来说，只要两个运算对象中的一个为真结果就为真。

逻辑与运算符和逻辑或运算符都是先求左侧运算对象的值再求右侧运算对象的值，当且仅当左侧运算对象无法确定表达式的结果时才会计算右侧运算对象的值。这种策略称为短路求值（short-circuit evaluation）。

- 对于逻辑与运算符来说，当且仅当左侧运算对象为真时才对右侧运算对象求值。
- 对于逻辑或运算符来说，当且仅当左侧运算对象为假时才对右侧运算对象求值。